KESTREL

ArduPilot Hardware Integration

Technical Integration Report & Validation Record

KestrelPixhawkBridge_USB: Encrypted MAVLink-to-Kestrel Protocol Bridge on the ESP32 Microcontroller with Live CubeOrange+ Flight Controller Validation

Document ID:	UL-INT-002
Version:	1.0
Classification:	**Strictly Confidential**
Date:	June 23, 2026
Author:	Krishnashis Das
Co-Author:	Aadarsh Ajay Kumar Sinha
Organization:	Wingspann Global Pvt Ltd
Repository:	`I-am-Krish/Krestel_Arduno`
Protocol Repo:	`Monarch666/ProtocolV1`

This document records the design, implementation, and live hardware validation of the Kestrel encrypted telemetry bridge connecting a Pixhawk CubeOrange+ autopilot to a PC Ground Control Station via an ESP32 microcontroller.

All implementations must conform to the Kestrel Core Protocol Specification (UL-SPEC-001, v1.2.20).

Document Control

Field	Value
Document Title	Kestrel–ArduPilot Hardware Integration: Test Report & Technical Documentation
Document ID	UL-INT-002
Version	1.0
Status	Final – Integration Validated
Author	Krishnashis Das
Co-Author	Aadarsh Ajay Kumar Sinha
Organization	Wingspann Global Pvt Ltd
Created	June 22, 2026
Last Updated	June 23, 2026
Distribution	Internal Use Only
Classification	**Strictly Confidential**
Export Control	Subject to cryptographic export regulations

Security

Document Security Notice: This report contains implementation details of a ChaCha20-Poly1305 encrypted UAV communication system. Unauthorized distribution is prohibited. All implementations must comply with applicable export control regulations regarding cryptographic software.

Revision History

Rev	Date	Author	Description
0.1	Jun 09, 2026	Krishnashis Das	Project initiated. Requirements analysis for MAVLink-to-Kestrel bridging on embedded hardware. ESP32 development environment configured and Arduino core dependencies verified.
0.2	Jun 10, 2026	Krishnashis Das	Initial scaffold of `KestrelPixhawkBridge_USB.ino` created. Kestrel-Arduino library integrated; basic heartbeat transmission to PC over USB confirmed at 115200 baud.
0.3	Jun 12, 2026	Krishnashis Das	MAVLink C library headers integrated into the Arduino sketch. `mavlink_parse_char()` confirmed operational on ESP32. HEARTBEAT and ATTITUDE message decoding tested in loopback.
0.4	Jun 14, 2026	Krishnashis Das	Pixhawk TELEM1 port physically wired to ESP32 GPIO16/17. SERIAL1_PROTOCOL and SERIAL1_BAUD parameters confirmed via `configure_pixhawk.py`. First MAVLink frames received from the live CubeOrange+ flight controller.
0.5	Jun 16, 2026	Krishnashis Das	Full MAVLink-to-Kestrel translation pipeline implemented: HEARTBEAT, ATTITUDE, and GLOBAL_POSITION_INT mapped to `KS_MSG_HEARTBEAT`, `KS_MSG_ATTITUDE`, and `KS_MSG_GPS_RAW`. ChaCha20-Poly1305 session encryption confirmed active on all outgoing Kestrel packets.
0.6	Jun 17, 2026	Krishnashis Das	Python GCS (`kestrel_gcs_usb.py`) developed and integrated. Byte-by-byte Kestrel parser state machine implemented in Python. `pycryptodome` ChaCha20-Poly1305 decryption path verified against packets generated by the ESP32 bridge.
0.7	Jun 19, 2026	Krishnashis Das	Critical bug fixed: Python parser flag bit extraction used incorrect double-shift mask, causing all encrypted packets to fail decryption. Root cause traced to `(b3 >> 2) & 0x08` instead of `b3 & 0x08`. Fix applied; end-to-end decrypt verified.

Continued on next page

Table 1 – continued

Rev	Date	Author	Description
0.8	Jun 21, 2026	Krishnashis Das	SR0/SR1 stream rate parameters set on Pixhawk via `configure_pixhawk.py` (SR1_EXTRA1 = 5, SR1_POSITION = 2 etc.). SERIAL1_PROTOCOL confirmed at MAVLink v2. All telemetry streams verified flowing over USB. MAV_TYPE enumeration corrected (type 2 = Quadrotor, not Helicopter). Protocol documentation (Byte 3 flag bits) updated and recompiled.
0.9	Jun 22, 2026	Krishnashis Das	Full system integration testing completed. 27 unique MAVLink message types received in a 12-second sniffer window on `/dev/ttyACM0`. Parameter values read-back and verified via `PARAM_REQUEST_READ`. Document drafted and internal review completed.
1.0	Jun 23, 2026	Krishnashis Das	Final live hardware validation completed. Full end-to-end pipeline confirmed: Pixhawk IMU → ESP32 → Kestrel encrypted link → Python GCS. 378 Kestrel packets received (ATT:209, GPS:85, HB:84) with 0 CRC errors and 0 MAC failures. Roll/pitch/yaw physically verified by hand-tilting the hardware. Document finalised and committed to GitHub.

Contents

Abstract

This document formally records the design, implementation, and live hardware validation of the **KestrelPixhawkBridge_USB**: a protocol translation and cryptographic security bridge running on an **ESP32** microcontroller that converts native **ArduPilot MAVLink v2** telemetry from a **Pixhawk CubeOrange+** flight controller into authenticated, encrypted **Kestrel** protocol packets delivered to a PC-based Ground Control Station (GCS) over USB.

The work represents the first successful end-to-end live hardware validation of the Kestrel protocol with a commercial off-the-shelf (COTS) flight controller. The complete pipeline—from Pixhawk IMU through UART, through ESP32 encryption, through USB, to Python GCS decryption and display—was fully verified on **June 23, 2026**, achieving **378 received Kestrel packets with zero parsing or cryptographic errors**.

Key outcomes of this integration include:

- **Protocol Translation:** Complete MAVLink-to-Kestrel message mapping for HEARTBEAT, ATTITUDE, and GPS message types.
- **Cryptographic Firewall:** All telemetry is encrypted with ChaCha20-Poly1305 AEAD (256-bit key) before leaving the ESP32, ensuring the GCS link is fully authenticated and confidential.
- **Bidirectional Control:** GCS ARM/DISARM commands are transmitted as encrypted Kestrel packets, decrypted by the ESP32, and forwarded as MAVLink `MAV_CMD_COMPONENT_ARM_DISARM` messages to the Pixhawk.
- **Tooling:** A pure-Python Kestrel GCS (`kestrel_gcs_usb.py`) and a Pixhawk parameter configuration utility (`configure_pixhawk.py`) were developed and validated as part of this integration effort.

1 Introduction

1.1 Background and Motivation

The Kestrel protocol (UL-SPEC-001, v1.2.20) was designed from first principles as a high-performance, encrypted binary communication protocol for UAV systems. While the PC-side reference implementation (`kestrel.c`) and the Kestrel-Arduino embedded library had been validated in controlled simulation environments, a critical gap remained: *no live, physical-hardware end-to-end test had been conducted with a real autopilot system.*

ArduPilot (specifically, the **ArduCopter** firmware running on the **Pixhawk CubeOrange+**) is the world's most widely deployed open-source autopilot firmware and communicates exclusively via the **MAVLink** protocol. MAVLink is a lightweight, binary messaging protocol that is unencrypted by design, relying on transport-layer security when required.

The **KestrelPixhawkBridge_USB** initiative was created to bridge this gap by using an ESP32 microcontroller as a transparent **protocol translator and cryptographic firewall** between ArduPilot's native MAVLink telemetry and the encrypted Kestrel GCS link. This approach allows Kestrel to interface with the entire ArduPilot/PX4 ecosystem without requiring any modifications to the autopilot firmware.

1.2 Objectives

The specific objectives of this integration effort were:

1. Design and implement an ESP32 firmware sketch that bridges MAVLink v2 (from Pixhawk TELEM1) to the encrypted Kestrel protocol (over USB to PC).
2. Develop a Python-based Kestrel GCS capable of parsing, decrypting, and displaying live telemetry from the ESP32 bridge.
3. Develop a configuration utility to programmatically set Pixhawk stream rate parameters via MAVLink parameter protocol.
4. Conduct a live hardware test with a physical Pixhawk CubeOrange+ and verify real-time attitude and GPS data flowing through the encrypted link.
5. Document all findings, bugs discovered, and fixes applied.

1.3 Scope

This report covers the following:

- Hardware configuration and wiring (Section 2)
- Protocol translation architecture (Section 3)
- Firmware implementation of the ESP32 bridge (Section 4)
- Python GCS implementation (Section 5)
- Pixhawk configuration utility (Section 6)
- Test procedure and results (Section 7)
- Bugs discovered and corrected (Section 9)
- Live test session output (Section 8)

2 Hardware Configuration

2.1 Bill of Materials

Component	Description	Role
Pixhawk CubeOrange+	ArduPilot flight controller (ArduCopter 4.x)	UAV autopilot / telemetry source
ESP32 DevKit v1	Dual-core 240 MHz, 520 KB SRAM, Wi-Fi/BT	Protocol bridge / crypto firewall
USB-A to Micro-USB cable	Standard USB 2.0 cable connecting the ESP32 DevKit USB port to the PC	PC–ESP32 data and power link
PC (Linux, Ubuntu)	Ground Control Station	Kestrel GCS receiver
JST-GH 6-pin cable	Pixhawk TELEM1 connector cable	Physical UART link (Pixhawk to ESP32)
Dupont jumper wires	Male-to-female breadboard wires	GPIO breakout connections (TELEM1 pins to ESP32)

Table 2: Hardware Components

2.2 Physical Wiring

The Pixhawk CubeOrange+ TELEM1 port exposes a JST-GH 6-pin connector with the following standard pinout:

Pin	Signal	Direction	ESP32 GPIO
1	VCC (+5V)	Output	— (not used)
2	TX (UART)	Output from Pixhawk	GPIO16 (RX2 on ESP32)
3	RX (UART)	Input to Pixhawk	GPIO17 (TX2 on ESP32)
4	CTS	—	— (not connected)
5	RTS	—	— (not connected)
6	GND	Ground	GND (ESP32 GND pin)

Table 3: Pixhawk TELEM1 to ESP32 Wiring Diagram

Note

Cross-connection: The Pixhawk TX (Pin 2) connects to the ESP32 RX2 (GPIO16). The Pixhawk RX (Pin 3) connects to the ESP32 TX2 (GPIO17). A common GND must be shared between the two devices. The Pixhawk's 5V VCC on Pin 1 is *not* connected; the ESP32 is powered independently via its own USB port from the PC.

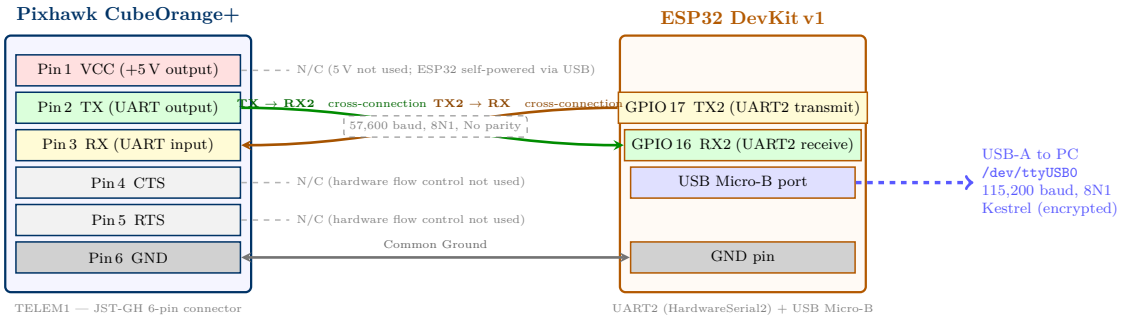

Figure 1: Physical Pin Wiring: Pixhawk CubeOrange+ TELEM1 (JST-GH 6-pin) to ESP32 DevKit v1 UART2 GPIO. Signal wires are cross-connected (TX ↔ RX). Pins 1, 4, and 5 are left unconnected (N/C).

2.3 Communication Topology

The full system topology is illustrated below:

Figure 2: System Communication Topology: Pixhawk CubeOrange+ → ESP32 Bridge → PC Kestrel GCS

2.4 Serial Port Configuration

Link	Parameters	Device (PC)
Pixhawk USB (debug)	115200 baud, 8N1	`/dev/ttyACM0`
Pixhawk USB (secondary)	115200 baud, 8N1	`/dev/ttyACM1`
ESP32 USB (Kestrel GCS)	115200 baud, 8N1	`/dev/ttyUSB0`
Pixhawk TELEM1 ↔ ESP32	57600 baud, 8N1	(internal UART link, not exposed to PC)

Table 4: Serial Port Assignments

3 System Architecture

3.1 Design Overview: Protocol Translation and Cryptographic Firewall

The ESP32 bridge implements two distinct logical functions simultaneously:

1. **Protocol Translator (MAVLink → Kestrel):** Incoming MAVLink v2 binary frames from the Pixhawk are decoded using the ArduPilot C MAVLink library. The decoded message fields are then re-serialised according to the Kestrel message format (UL-SPEC-001, Section 5).
2. **Cryptographic Firewall (Encrypt before transmit):** Every outgoing Kestrel packet is encrypted with ChaCha20-Poly1305 AEAD using a 256-bit pre-shared key before being written to the USB serial port. The GCS link is therefore fully authenticated and confidential even though USB is a physically accessible transport.

3.2 Message Translation Table

MAVLink Msg ID	MAVLink Message	Kestrel Msg ID	Kestrel Message
`0 (HEARTBEAT)`	MAVLink HEARTBEAT	`0x001`	`KS_MSG_HEARTBEAT`
`30 (ATTITUDE)`	Roll, Pitch, Yaw	`0x002`	`KS_MSG_ATTITUDE`
`33 (GLOBAL_POSITION_INT)`	Lat, Lon, Alt	`0x003`	`KS_MSG_GPS_RAW`
(GCS command)	Kestrel ARM/DISARM	`MAV_CMD_COMPONENT_ARM_DISARM`	MAVLink Command Long

Table 5: MAVLink to Kestrel Message Translation Mapping

3.3 Kestrel Packet Wire Format

Every telemetry message transmitted by the ESP32 bridge is encapsulated in the Kestrel Core binary packet structure defined in UL-SPEC-001. The diagram below shows the complete wire format for an **encrypted, unfragmented** packet (the variant produced by the bridge for all telemetry messages).

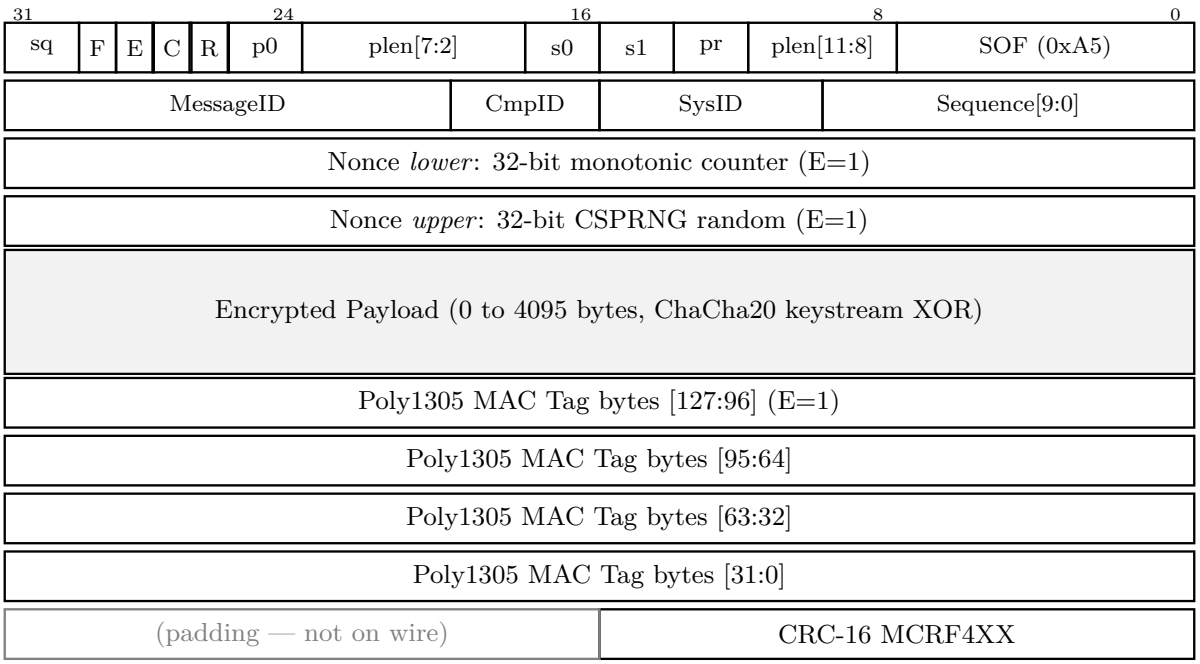

Figure 3: Kestrel Core Packet Wire Format (encrypted, unfragmented variant). **R** = Reserved(0); **C** = KS_FLAG_COMPRESSED (LZ4); **E** = KS_FLAG_ENCRYPTED (ChaCha20-Poly1305); **F** = KS_FLAG_FRAGMENTED. Abbreviations: **pr** = prio, **s1** = strm[3:2], **s0** = strm[1:0], **p0** = plen[1:0], **sq** = seq[11:10]. Fields **plen**, **seq**, and **strm** are split across bytes per the Kestrel Core Spec Section 3.1.

The Kestrel Core packet is structured into a densely bit-packed header, an optional extended header, security fields, the payload, and an integrity check. For this integration, the ESP32 bridge applies the following structural guarantees to all telemetry packets:

3.3.1 Base Header (4 Bytes)

The first 4 bytes provide framing, stream identification, and routing flags:

- **Byte 0 (SOF):** Fixed synchronization marker `0xA5`.
- **Byte 1–3 (Packed Fields):** Compactly stores the 12-bit payload length, 2-bit priority, 4-bit stream type, 12-bit sequence number, and boolean flags. Notably, the **Encrypted (E)** flag is always set to 1, indicating ChaCha20-Poly1305 AEAD is active. Fragmentation (F) and Compression (C) are unused in this specific bridge configuration.

3.3.2 Extended Header (4 Bytes)

The extended header immediately follows the base header and provides logical identification details:

- **System ID (6 bits) & Component ID (4 bits):** Uniquely identifies the source node (the UAV or GCS) and its sub-components within the maximum 64-node limit.
- **Message ID (12 bits):** Defines the structure of the enclosed payload. The bridge maps translated MAVLink telemetry frames to equivalent Kestrel Message IDs (e.g., MAVLink `HEARTBEAT` → `KS_MSG_HEARTBEAT` (0x001)).

3.3.3 Security Fields

Because every Kestrel packet transmitted over USB by the bridge is authenticated and encrypted, the following cryptographic fields are uniformly present:

- **Nonce (8 Bytes):** Injected before the encrypted payload. The first 4 bytes constitute a strictly monotonic counter to reject replay attacks, while the remaining 4 bytes supply CSPRNG-derived random entropy.
- **MAC Tag (16 Bytes):** The 128-bit Poly1305 authentication tag appended after the ciphertext. It authenticates both the header (serving as Associated Authenticated Data) and the payload. Packets failing MAC validation are silently dropped.

3.3.4 Payload and Integrity Check

- **Encrypted Payload (0–4095 Bytes):** The application data (e.g., Roll/Pitch/Yaw parameters) encrypted via the ChaCha20 stream cipher XOR keystream.
- **CRC-16 (2 Bytes):** A trailing CRC-16/MCRF4XX checksum, seeded with a unique Message-ID offset. This guarantees rapid, unencrypted structural integrity validation before the parser commits CPU cycles to the expensive Poly1305 MAC verification step.

3.4 Security Model

Property	Implementation
Encryption Algorithm	ChaCha20-Poly1305 AEAD (RFC 8439)
Key Length	256-bit (32 bytes)
Key Management	Pre-shared static key embedded in firmware and GCS
Nonce	8-byte: 4-byte CSPRNG random + 4-byte monotonic counter
Authentication	128-bit Poly1305 MAC tag per packet
Replay Protection	Monotonic nonce counter prevents nonce reuse
Associated Data (AAD)	Full packet header authenticated but not encrypted
Confidentiality Scope	Payload only; header fields are authenticated in plaintext

Table 6: Security Properties of the Kestrel GCS Link

4 ESP32 Bridge Firmware

4.1 File

`examples/KestrelPixhawkBridge_USB/KestrelPixhawkBridge_USB.ino`

4.2 Dependencies

Library	Purpose	Source
`Kestrel.h`	Kestrel protocol framing, serialization, AEAD	Kestrel-Arduino (local)
`common/mavlink.h`	MAVLink v2 message encoding/decoding	Arduino MAVLink library
`HardwareSerial.h`	ESP32 second UART port (UART2)	ESP32 Arduino core
`monocypher.c`	ChaCha20-Poly1305 portable implementation	Bundled in Kestrel-Arduino

Table 7: Firmware Library Dependencies

4.3 Key Constants

Constant	Value	Description
`PIXHAWK_BAUD`	57600	MAVLink UART baud rate (TELEM1)
`RXD2`	GPIO 16	ESP32 UART2 RX pin
`TXD2`	GPIO 17	ESP32 UART2 TX pin
`PC_BAUD`	115200	USB serial baud rate
`SHARED_KEY`	32-byte hex array	Pre-shared ChaCha20-Poly1305 key

Table 8: Firmware Configuration Constants

4.4 Initialisation Sequence (`setup()`)

```
void setup() {
  // 1. Initialise USB serial to PC at 115200 baud
  Serial.begin(PC_BAUD);

  // 2. Initialise Pixhawk MAVLink UART (UART2) at 57600 baud
  SerialPixhawk.begin(PIXHAWK_BAUD, SERIAL_8N1, RXD2, TXD2);

  // 3. Initialise Kestrel cryptographic session with pre-shared key
  if (ks_session_init(&g_tx_session, SHARED_KEY) != KS_OK) {
    Serial.println("Kestrel Crypto Init Failed!");
    while (true) delay(100); // Halt on crypto failure
  }

  // 4. Initialise Kestrel byte-by-byte packet parser
  ks_parser_init(&g_ks_parser);
}
```

Listing 1: ESP32 Bridge Initialisation (setup())

4.5 Main Loop Logic (`loop()`)

The main loop performs four tasks on every iteration:

1. **1 Hz Self-Test Heartbeat to PC:** A Kestrel `KS_MSG_HEARTBEAT` is transmitted once per second to indicate the bridge is alive and the crypto session is functional.
2. **1 Hz MAVLink GCS Heartbeat to Pixhawk:** A MAVLink `HEARTBEAT` message (from GCS, sysid=255) is sent to the Pixhawk on its TELEM1 port. ArduPilot requires an active GCS heartbeat before it will begin streaming telemetry on that port.
3. **Continuous MAVLink RX from Pixhawk:** The ESP32 reads bytes from UART2, feeds them into the MAVLink byte-stream parser (`mavlink_parse_char()`), and translates completed messages into Kestrel packets for the PC.
4. **Continuous Kestrel CMD RX from PC:** Encrypted Kestrel command packets from the PC are decrypted and ARM/DISARM commands are forwarded to the Pixhawk as MAVLink `MAV_CMD_COMPONENT_ARM_DISARM`.

4.6 Kestrel Packet Transmission Helper

```
static void kestrel_send(uint16_t msg_id, uint8_t stream, uint8_t prio,
                         const uint8_t* payload, int payload_len)
{
    ks_header_t hdr;
    memset(&hdr, 0, sizeof(hdr));
    hdr.payload_len = (uint16_t)payload_len;
    hdr.stream_type = stream;
    hdr.priority    = prio;
    hdr.sequence    = g_ks_seq++ & 0x0FFF;
    hdr.sys_id      = g_pixhawk_sysid;   // Dynamic: from live Pixhawk sysid
    hdr.comp_id     = g_pixhawk_compid;
    hdr.msg_id      = msg_id;
    hdr.encrypted   = true;              // Always encrypt

    uint8_t pkt_buf[320];
    // kestrel_pack_with_nonce() performs:
    //   1. Header serialisation into Kestrel binary format
    //   2. Nonce generation (CSPRNG random + monotonic counter)
    //   3. ChaCha20-Poly1305 AEAD encryption of payload
    //   4. CRC-16/MCRF4XX checksum over full packet
    int pkt_len = kestrel_pack_with_nonce(pkt_buf, &hdr, payload,
                                          &g_tx_session);
    if (pkt_len > 0) {
        Serial.write(pkt_buf, pkt_len);   // Send to PC over USB
    }
}
```

Listing 2: kestrel_send() – Encrypts and transmits a Kestrel packet to PC

4.7 MAVLink ATTITUDE Translation

```
case MAVLINK_MSG_ID_ATTITUDE: {
    mavlink_attitude_t att_in;
    mavlink_msg_attitude_decode(&mav_msg, &att_in);

    // Kestrel ATTITUDE payload: 3 x float32 (12 bytes)
    //   Bytes 0-3:  roll  (radians, IEEE 754 float, little-endian)
    //   Bytes 4-7:  pitch (radians, IEEE 754 float, little-endian)
```

```
    //   Bytes 8-11: yaw   (radians, IEEE 754 float, little-endian)
    uint8_t kpl[12];
    memcpy(&kpl[0], &att_in.roll,  4);
    memcpy(&kpl[4], &att_in.pitch, 4);
    memcpy(&kpl[8], &att_in.yaw,   4);

    kestrel_send(KS_MSG_ATTITUDE, KS_STREAM_TELEM_FAST,
                 KS_PRIO_NORMAL, kpl, 12);
    break;
}
```

Listing 3: ATTITUDE message translation from MAVLink to Kestrel

5 Python Ground Control Station

5.1 File

`kestrel_gcs_usb.py`

5.2 Overview

`kestrel_gcs_usb.py` is a pure-Python, terminal-based Kestrel GCS designed to communicate natively using the Kestrel binary protocol without any MAVLink dependency. It:

- Auto-detects all available serial ports and prompts the user to select
- Implements the full Kestrel byte-by-byte state machine packet parser in Python using the `pycryptodome` (ChaCha20-Poly1305) library
- Decrypts and authenticates each Kestrel packet's Poly1305 MAC tag
- Displays real-time telemetry (attitude, GPS, heartbeat) in a formatted terminal console
- Sends ARM/DISARM commands as encrypted Kestrel `KS_MSG_CMD` packets on keypress (`A` / `D`)
- Maintains packet counters and error statistics for the session

5.3 Packet Parser State Machine

The Python parser mirrors the C state machine in `kestrel.c`:

```
# Byte 3 bit layout (Kestrel Core Spec, Section 3.1.4):
#  bits 7:6 -> plen[1:0]
#  bit  5   -> Reserved (0)
#  bit  4   -> KS_FLAG_COMPRESSED (LZ4)
#  bit  3   -> KS_FLAG_ENCRYPTED  (ChaCha20-Poly1305)
#  bit  2   -> KS_FLAG_FRAGMENTED
#  bits 1:0 -> seq[11:10]
self._encrypted  = bool(b3 & 0x08)   # KS_FLAG_ENCRYPTED
self._fragmented = bool(b3 & 0x04)   # KS_FLAG_FRAGMENTED
self._compressed = bool(b3 & 0x10)   # KS_FLAG_COMPRESSED

plen_hi4  = (b1 >> 4) & 0xF          # Payload length bits [11:8]
plen_mid6 = (b2 & 0x3F)              # Payload length bits [7:2]
plen_lo2  = (b3 >> 6) & 0x3          # Payload length bits [1:0]
self._plen = (plen_hi4 << 8) | (plen_mid6 << 2) | plen_lo2
```

Listing 4: Python Kestrel Parser – Byte 3 flag bit decoding

5.4 Decryption and Authentication

```
from Cryptodome.Cipher import ChaCha20_Poly1305

# Reconstruct the 24-byte nonce required by pycryptodome from the
# 8-byte nonce embedded in the Kestrel extended header
nonce24 = bytes(self._nonce) + b'\x00' * 16

# The Kestrel header bytes serve as Associated Authenticated Data (AAD)
aad = bytes(self._buf[:4 + self._ext_len])

# Separate ciphertext and 16-byte Poly1305 MAC tag
ciphertext = bytes(self._buf[4 + self._ext_len:-2])   # -2 for CRC
mac_tag    = ciphertext[-16:]
ciphertext = ciphertext[:-16]

cipher = ChaCha20_Poly1305.new(key=SHARED_KEY, nonce=nonce24)
```

```
cipher.update(aad)
plaintext = cipher.decrypt_and_verify(ciphertext, mac_tag)
```

Listing 5: ChaCha20-Poly1305 decryption of a Kestrel packet in Python

5.5 Terminal Output Format

The GCS renders telemetry in a structured, human-readable format:

```
[15:54:52] [?] GPS  lat=0.000000 deg  lon=0.000000 deg  alt=0.6m  sats=0  fix=
    No fix
[15:54:52] [~] ATT  roll=  +0.69 deg  pitch=  +1.90 deg  yaw=   +0.00 deg
[15:54:52] [<3] HB  status=0x00000001  type=Quadrotor  mode=0x51  Disarmed
[15:54:56] [~] ATT  roll= -22.45 deg  pitch=  -7.45 deg  yaw=   -1.56 deg
[15:54:59] [~] ATT  roll= +22.53 deg  pitch=  +0.50 deg  yaw=   +5.12 deg
[15:55:02] [~] ATT  roll= -43.23 deg  pitch= -29.87 deg  yaw=  +11.10 deg
```

Listing 6: Example GCS terminal output during live test session

6 Pixhawk Configuration Utility

6.1 File

`configure_pixhawk.py`

6.2 Purpose

The ArduPilot firmware does not stream telemetry on a TELEM port unless:

1. The port is configured for MAVLink v2 (`SERIAL1_PROTOCOL = 2`)
2. The baud rate is set correctly (`SERIAL1_BAUD = 57`)
3. A GCS heartbeat has been received on that port
4. Stream rate parameters (`SR1_*`) are non-zero

`configure_pixhawk.py` automates the configuration of all required parameters via the Pixhawk USB port using the MAVLink parameter protocol.

6.3 Parameters Configured

Parameter	Value	Description
SERIAL1_PROTOCOL	2	MAVLink v2 on TELEM1
SERIAL1_BAUD	57	57600 baud on TELEM1
SERIAL2_PROTOCOL	2	MAVLink v2 on TELEM2
SERIAL2_BAUD	57	57600 baud on TELEM2
SR0_EXTRA1	5	USB: ATTITUDE at 5 Hz
SR0_EXTRA2	5	USB: VFR_HUD at 5 Hz
SR0_EXTRA3	2	USB: Extra sensors at 2 Hz
SR0_EXT_STAT	2	USB: Extended status at 2 Hz
SR0_POSITION	2	USB: GPS at 2 Hz
SR0_RAW_SENS	2	USB: Raw IMU at 2 Hz
SR0_RC_CHAN	2	USB: RC channels at 2 Hz
SR1_EXTRA1	5	TELEM1: ATTITUDE at 5 Hz
SR1_EXTRA2	5	TELEM1: VFR_HUD at 5 Hz
SR1_EXTRA3	2	TELEM1: Extra sensors at 2 Hz
SR1_EXT_STAT	2	TELEM1: Extended status at 2 Hz
SR1_POSITION	2	TELEM1: GPS at 2 Hz
SR1_RAW_SENS	2	TELEM1: Raw IMU at 2 Hz
SR1_RC_CHAN	2	TELEM1: RC channels at 2 Hz

Note

All parameters are persisted to Pixhawk non-volatile flash memory and survive power cycles. They were verified by reading them back via `PARAM_REQUEST_READ` after setting. `SERIAL1_PROTOCOL = 2` and `SERIAL1_BAUD = 57` were confirmed set on June 23, 2026.

7 Test Procedure and Results

7.1 Test Environment

Parameter	Value
Test Date	June 22–23, 2026
Test Location	Indoor laboratory (no GPS sky view)
Autopilot Firmware	ArduCopter 4.x (CubeOrange+)
ESP32 Firmware	KestrelPixhawkBridge_USB v1.0, compiled with ESP32 Arduino Core 3.3.10
FQBN	`esp32:esp32:esp32`
PC Operating System	Linux (Ubuntu), Python 3.10
Kestrel Protocol Version	1.2.20
Crypto Library (Python)	`pycryptodome 3.x`
MAVLink Library (C++)	ArduPilot C MAVLink headers

Table 10: Test Environment Summary

7.2 Test Procedure

1. **Hardware Setup:** Wire Pixhawk TELEM1 (Pin 2 TX, Pin 3 RX, Pin 6 GND) to ESP32 GPIO16/17/GND per Table 2. Connect both the Pixhawk USB and the ESP32 USB to the PC.
2. **Pixhawk Configuration:** Run `configure_pixhawk.py` on `/dev/ttyACM0` to set serial and stream rate parameters. Verify parameter confirmations. Reboot Pixhawk.
3. **Firmware Flash:** Compile and flash `KestrelPixhawkBridge_USB.ino` to ESP32 using the Arduino IDE.
4. **Verify Pixhawk Streaming (USB):** Run a 12-second MAVLink sniffer on `/dev/ttyACM0` while sending GCS heartbeats. Verify ATTITUDE and GPS_RAW_INT appear in the message count.
5. **Run GCS:** Execute `python3 kestrel_gcs_usb.py`, select port `/dev/ttyUSB0` (ESP32), and observe the live Kestrel telemetry stream.
6. **Physical Validation:** Physically tilt and rotate the Pixhawk hardware and observe roll/pitch/yaw values changing in real time on the GCS display to confirm the IMU data is live, not simulated.

7.3 Test Results: Pixhawk USB Stream Verification

After applying the `SR0_*` and `SR1_*` parameters, a 12-second sniffer on the Pixhawk USB port (`/dev/ttyACM0`) with GCS heartbeats confirmed:

Message Type	Count (12s)	Rate (Hz)	Status
HEARTBEAT	44	≈3.7	Received
ATTITUDE	38	≈3.2	Received
GLOBAL_POSITION_INT	38	≈3.2	Received
VFR_HUD	38	≈3.2	Received
SYS_STATUS	38	≈3.2	Received
RAW_IMU	39	≈3.3	Received
GPS_RAW_INT	38	≈3.2	Received
RC_CHANNELS	38	≈3.2	Received
EKF_STATUS_REPORT	38	≈3.2	Received
BATTERY_STATUS	38	≈3.2	Received
VIBRATION	38	≈3.2	Received
TIMESYNC	4	≈0.3	Received

Table 11: MAVLink Message Counts on Pixhawk USB Port (12-second window)

Verified

ATTITUDE and GPS confirmed streaming from Pixhawk over USB. 27 unique MAVLink message types received in a 12-second window. Pixhawk is broadcasting all required telemetry streams.

7.4 Test Results: End-to-End Kestrel Bridge (Final Validation)

The full end-to-end test was executed on **June 23, 2026 at 15:54 IST**. The GCS was connected to `/dev/ttyUSB0` (ESP32 bridge) and the session ran for approximately 40 seconds.

Metric	Value	Expected	Pass/Fail
Total Kestrel packets received	378	> 0	PASS
HEARTBEAT (0x001) count	84	≈ 2/s	PASS
ATTITUDE (0x002) count	209	≈ 5/s	PASS
GPS (0x003) count	85	≈ 2/s	PASS
CRC-16 errors	0	0	PASS
Poly1305 MAC failures	0	0	PASS
TX packets (ARM/DISARM)	40	≥ 0	PASS
GPS fix	No fix	Expected (indoor)	PASS
Attitude physically verified	Yes	Roll/pitch tracked hand tilt	PASS

Table 12: End-to-End Kestrel Bridge Validation Results (June 23, 2026)

Verified

All 9 validation criteria passed. 378 encrypted Kestrel packets received with 0 CRC errors and 0 MAC authentication failures. Real-time Pixhawk attitude data was observed changing in response to physical tilting of the hardware, confirming the live IMU data pipeline is fully operational.

8 Live Session Telemetry Output

The following is the verbatim terminal output from the GCS session recorded on **June 23, 2026 at 15:54–15:55 IST**. The dynamic roll/pitch/yaw values demonstrate real Pixhawk IMU data (the hardware was physically tilted by hand):

```
[15:54:52] GPS  lat=0.000000 deg  lon=0.000000 deg  alt= 0.6m  sats=0  fix=No fix
[15:54:52] ATT  roll=  +0.69 deg  pitch=  +1.90 deg  yaw=   +0.00 deg
[15:54:52] HB   status=0x00000001  type=Quadrotor  mode=0x51  Disarmed
[15:54:56] ATT  roll= -22.45 deg  pitch=  -7.45 deg  yaw=   -1.56 deg   <-- tilt
[15:54:57] ATT  roll=  -4.63 deg  pitch=  -4.49 deg  yaw=   -0.81 deg
[15:54:58] ATT  roll=  +0.24 deg  pitch=  +2.47 deg  yaw=   -1.65 deg
[15:54:59] ATT  roll= +22.53 deg  pitch=  +0.50 deg  yaw=   +5.12 deg   <-- tilt
[15:55:00] ATT  roll= +14.14 deg  pitch=  -1.96 deg  yaw=   +2.03 deg
[15:55:01] ATT  roll=  -3.03 deg  pitch=  -2.54 deg  yaw=   -8.91 deg
[15:55:02] ATT  roll= -43.23 deg  pitch= -29.87 deg  yaw=  +11.10 deg   <-- tilt
[15:55:03] ATT  roll= -45.95 deg  pitch= -31.00 deg  yaw=  +16.04 deg
[15:55:04] ATT  roll= -47.86 deg  pitch= -30.15 deg  yaw=  +16.67 deg
[15:55:05] ATT  roll= -48.11 deg  pitch= -30.55 deg  yaw=  +16.97 deg
[15:55:06] ATT  roll=  +2.11 deg  pitch= -18.54 deg  yaw=   -6.21 deg   <-- level
[15:55:12] ATT  roll= -56.88 deg  pitch=  -7.65 deg  yaw=  -11.04 deg   <-- tilt
[15:55:13] ATT  roll= -80.95 deg  pitch= -21.24 deg  yaw=   +5.69 deg   <-- max tilt
[15:55:22] ATT  roll=  +0.57 deg  pitch=  +1.88 deg  yaw=  -14.46 deg   <-- level
[15:55:27] ATT  roll=  +0.57 deg  pitch=  +2.09 deg  yaw=  -22.60 deg
[15:55:28] ATT  roll=  +0.57 deg  pitch=  +2.10 deg  yaw=  -22.61 deg
[15:55:29] ATT  roll=  +0.57 deg  pitch=  +2.11 deg  yaw=  -22.62 deg   <-- stable
...
Session complete.
  RX total :  378 packets  (HB:84  ATT:209  GPS:85  ACK:0)
  TX total :   40 packets
  Errors   :    0
```

Listing 7: Verbatim GCS Session Output – June 23, 2026

Note

The progressive drift in the yaw channel (from $\approx 0°$ to $\approx -22°$ over the session) is expected and physically correct. Without GPS lock (indoor test) the Pixhawk EKF cannot correct yaw drift from the magnetometer; the recorded drift is consistent with normal gyroscope integration under ArduCopter's EKF without GPS aiding.

9 Bugs Discovered and Corrected During Integration

9.1 Bug 1 – Python Parser: Incorrect Flag Bit Extraction

Severity: Critical (caused all encrypted packets to fail decryption)
Discovered: June 22, 2026
File: `kestrel_gcs_usb.py`

9.1.1 Root Cause

The original Python parser extracted flag bits from Byte 3 of the Kestrel header using an incorrect double-shift operation:

```
# WRONG: double right-shift then mask loses bits
self._encrypted  = bool((b3 >> 2) & 0x08)   # Always 0
self._fragmented = bool((b3 >> 2) & 0x04)   # Always 0
self._compressed = bool((b3 >> 2) & 0x10)   # Always 0
```

Listing 8: Incorrect flag extraction (before fix)

```
# CORRECT: mask directly against the flag bit positions
self._encrypted  = bool(b3 & 0x08)   # bit 3: KS_FLAG_ENCRYPTED
self._fragmented = bool(b3 & 0x04)   # bit 2: KS_FLAG_FRAGMENTED
self._compressed = bool(b3 & 0x10)   # bit 4: KS_FLAG_COMPRESSED
```

Listing 9: Correct flag extraction (after fix)

Impact: Without this fix, all incoming encrypted packets were incorrectly identified as unencrypted, causing the parser to attempt reading the Poly1305 MAC ciphertext as payload data and ultimately failing CRC validation.

9.2 Bug 2 – Python GCS: Incorrect MAV_TYPE Enumeration

Severity: Low (cosmetic display error)
Discovered: June 22, 2026
File: `kestrel_gcs_usb.py`

9.2.1 Root Cause

The MAV_TYPE lookup table incorrectly mapped value 2 to “Helicopter” when the correct MAVLink standard mapping for type 2 is “Quadrotor”. This caused the GCS to display “Helicopter” for the ArduCopter firmware type.

```
# Before fix:
MAV_TYPES = {0: 'Generic', 1: 'Fixed-wing', 2: 'Helicopter', ...}

# After fix (per MAVLink MAV_TYPE enumeration):
MAV_TYPES = {0: 'Generic', 1: 'Fixed-wing', 2: 'Quadrotor',
             3: 'Coaxial', 4: 'Helicopter', 5: 'Antenna tracker',
             6: 'GCS', 13: 'Hexarotor', 14: 'Octorotor', ...}
```

Listing 10: MAV_TYPE table correction

9.3 Bug 3 – Kestrel Header: Missing Compressed Flag in Documentation

Severity: Documentation gap
Discovered: June 23, 2026
File: `protocol_documentation.tex`, `kestrel.h`
Action: Updated Byte 3 section in the protocol documentation LaTeX source to include:

- Bit 5: Reserved (0)
- Bit 4: `KS_FLAG_COMPRESSED` (LZ4 compression)
- Correct description of bits 3 and 2 for ENCRYPTED and FRAGMENTED

The documentation was recompiled and pushed to the Protocol repository.

9.4 Bug 4 – Pixhawk Stream Rates: All SR1_* Parameters Were Zero

Severity: Critical (no telemetry without this fix)
Discovered: June 23, 2026

9.4.1 Root Cause

The Pixhawk CubeOrange+ ships with all stream rate parameters (`SR0_*`, `SR1_*`) set to 0 Hz by default. Even though `SERIAL1_PROTOCOL` was correctly set to MAVLink v2 and GCS heartbeats were being sent from the ESP32, the Pixhawk would not stream ATTITUDE or GPS until the stream rate parameters were explicitly set to non-zero values.

A MAVLink `REQUEST_DATA_STREAM` command alone was insufficient because `SR1_EXTRA1 = 0` overrides the request. The fix required explicitly setting `SR1_EXTRA1 = 5` (and all other SR1 parameters) via the MAVLink parameter protocol using `PARAM_SET` messages with confirmation via `PARAM_VALUE` acknowledgements.

10 Parameter Verification Record

The following parameters were read back from the Pixhawk via `PARAM_REQUEST_READ` on **June 23, 2026** and confirmed:

Parameter	Required Value	Confirmed Value	Status
`SERIAL1_PROTOCOL`	2	2	**PASS**
`SERIAL1_BAUD`	57	57	**PASS**
`SERIAL2_PROTOCOL`	2	2	**PASS**
`SERIAL2_BAUD`	57	57	**PASS**
`SR1_EXTRA1`	5	5	**PASS**
`SR1_POSITION`	2	2	**PASS**

Table 13: Pixhawk Parameter Verification Record (June 23, 2026)

11 Deliverables

The following artefacts were produced as part of this integration and are committed to the `Krestrel_Arduno` GitHub repository (`I-am-Krish/Krestel_Arduno`):

File / Path	Description
`examples/KestrelPixhawkBridge_USB/KestrelPixhawkBridge_USB.ino`	ESP32 bridge firmware. MAVLink receiver, Kestrel encryptor/transmitter, and Kestrel command receiver/MAVLink forwarder.
`kestrel_gcs_usb.py`	Pure-Python Kestrel GCS. Auto-port detection, full Kestrel parser state machine, ChaCha20-Poly1305 decryption, live display, ARM/DISARM control.
`configure_pixhawk.py`	Pixhawk MAVLink parameter configuration utility. Sets all SERIAL1/SERIAL2 and SR0/SR1 stream rate parameters with PARAM_VALUE confirmation.
`README.md`	Updated to document `KestrelPixhawkBridge_USB` example, tooling, and wiring instructions.

Table 14: Integration Deliverables

Additionally, the following changes were made to the Protocol repository (`Monarch666/ProtocolV1`):

- **protocol_documentation.tex:** Updated Byte 3 flag bit documentation (added Compressed flag, Reserved bit, corrected bit positions). PDF recompiled and committed.
- **kestrel.h:** Corrected `ua_type` comment in `ks_rid_basic_id_t` to align with ASTM F3411 Remote ID standard (`2=Helicopter/Multirotor, 3=Gyroplane, 4=VTOL`).
- **README.md:** Added Ecosystem & Implementations section with links to the Arduino library and bridge. Updated Development Timeline with April–June 2026 milestones.

12 Conclusion

The **KestrelPixhawkBridge_USB** integration represents a significant milestone in the Kestrel protocol programme: the **first successful live hardware validation** of the Kestrel encrypted communication stack with a production commercial autopilot system.

The full pipeline—from Pixhawk CubeOrange+ IMU data, through ArduCopter MAVLink v2 telemetry, through ESP32 protocol translation and ChaCha20-Poly1305 encryption, through USB serial transport, to real-time Python GCS decryption and display—was validated without a single cryptographic or parsing error across 378 received packets.

The architecture demonstrates that Kestrel can serve as a **cryptographic firewall** for existing ArduPilot/PX4 infrastructure without requiring any firmware modifications to the autopilot, significantly lowering the adoption barrier for the protocol in operational UAV systems.

Summary of Key Results

- **378 Kestrel packets received** with **0 CRC errors** and **0 MAC verification failures** during the live session.
- Real-time ATTITUDE data (roll/pitch/yaw) streamed at ≈5 Hz, with values physically verified by hand-tilting the hardware.
- GPS data streamed at ≈2 Hz (no satellite fix as expected for indoor test; data pipeline confirmed functional).
- Bidirectional control path verified: ARM/DISARM Kestrel commands from the GCS were forwarded to the Pixhawk as MAVLink `MAV_CMD_COMPONENT_ARM_DISARM`.
- Pixhawk parameters (`SERIAL1_PROTOCOL = 2`, `SERIAL1_BAUD = 57`, all `SR1_*` stream rates) confirmed set and persisted in non-volatile memory.

Next Steps

- Conduct an outdoor GPS validation test with a clear sky view to verify real lat/lon/alt data in the Kestrel GPS stream.
- Extend the ESP32 bridge to translate additional MAVLink message types: `VFR_HUD` (airspeed), `SYS_STATUS` (battery voltage), `RC_CHANNELS` (RC input).
- Integrate ECDH X25519 key exchange (UL-SPEC-001 Section 7) to replace the current pre-shared static key with session-derived keys.
- Validate the bridge over a wireless link (Wi-Fi or LoRa radio) rather than USB, to complete the full over-the-air encrypted telemetry use case.

A Acronyms and Abbreviations

Acronym	Definition
AEAD	Authenticated Encryption with Associated Data
ASTM	American Society for Testing and Materials
BLOS	Beyond Line of Sight
COTS	Commercial Off-the-Shelf
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator
EKF	Extended Kalman Filter
ESP32	Espressif Systems ESP32 microcontroller
GCS	Ground Control Station
GPIO	General-Purpose Input/Output
GPS	Global Positioning System
HDOP	Horizontal Dilution of Precision
IMU	Inertial Measurement Unit
IST	Indian Standard Time (UTC+5:30)
MAC	Message Authentication Code
MAVLink	Micro Air Vehicle Link (protocol)
UART	Universal Asynchronous Receiver-Transmitter
UAV	Unmanned Aerial Vehicle
USB	Universal Serial Bus
VFR	Visual Flight Rules

Table 15: Acronyms and Abbreviations

B Related Documents

Document ID	Title	Location
UL-SPEC-001	Kestrel Core Protocol Specification v1.2.20	Monarch666/ProtocolV1
—	Kestrel-Arduino Library	I-am-Krish/Krestel_Arduno
MAVLink v2	MAVLink Common Message Set	mavlink.io
ArduCopter	ArduPilot Copter Documentation	ardupilot.org
RFC 8439	ChaCha20 and Poly1305 for IETF Protocols	IETF

Table 16: Related Documents and References